Software Requirements Specification

for

Blog Management System API

Version 2.0

Prepared by Md Ismile Hosen Siam

August , 2026

Table of Contents

Table of Contents ..... **ii**

1. Introduction **1**

1.1 Purpose 1

1.2 Project Scope 1

1.3 References 2

2. Overall Description **2**

2.1 Product Perspective 2

2.2 Product Features 3

2.3 User Classes and Characteristics 4

2.4 Operating Environment 5

2.5 Design and Implementation Constraints 6

2.6 Assumptions and Dependencies 7

3. System Requirements **7**

3.1 Functional Requirements 7

3.2 Non-Functional Requirements 19

3.3 Interface Requirements 20

4. System Architecture **21**

4.1 Project Structure 21

4.2 Technology Stack 22

Appendix A: Glossary **23**

Appendix B: Analysis Models **24**

Use Case Diagram 24

Account State Diagram 25

Data Flow Diagram Level 0 25

Data Flow Diagram Level 1 26

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the **Blog Management System API**, Version 1.0. This Software Requirements Specification (SRS) covers the complete backend system of the application, which is designed to allow users to register, log in, and manage blog content through a RESTful API.

The scope of this document includes the core functional modules of the system — **authentication, article management, category management, comment management, and user management** — along with the non-functional aspects that support these modules, including **security, performance, scalability, caching, logging, testing, containerization (Docker), CI/CD, load balancing, and database replication**.

This SRS describes the backend system in its entirety, as the project does not include a client-facing frontend within its current scope; all functionality is exposed and consumed via REST API endpoints.

1.2 Project Scope

The software specified in this document is the **Blog Management System API**, a backend REST API that enables users to register, authenticate, and manage blog content such as articles and comments. The system supports two primary user roles: **registered users**, who can create, update, and delete their own articles, comment on articles, and reply to comments, and **administrators**, who have elevated permissions to manage users, categories, and system-wide content.

The primary objective of this system is to provide a secure, scalable, and maintainable backend platform for blog-based content management, following industry-standard practices in software architecture and deployment. Key goals of the system include:

- Providing secure user authentication and role-based access control.
- Enabling efficient creation, retrieval, updating, and deletion (CRUD) of articles and comments.
- Ensuring system reliability and performance through caching, logging, and load balancing.
- Supporting scalability via database replication and containerized deployment (Docker).
- Maintaining code quality and deployment consistency through automated testing and CI/CD pipelines.

This project is developed as a demonstration of backend engineering capability, showcasing practical implementation of authentication systems, RESTful API design, and production-grade deployment practices. It is intended to reflect real-world backend development standards applicable to scalable content management platforms, and serves as a reference implementation of secure and maintainable API architecture.

No separate vision and scope document exists for this project; this SRS serves as the sole specification of scope for the current release (Version 1.0).

1.3 References

- Project guidelines provided by the client.
- IEEE Recommended Practice for Software Requirements Specifications (IEEE 29148).
- OpenAPI Specification https://swagger.io/specification/
- Express.js Documentation: https://expressjs.com/
- Node.js Documentation: https://nodejs.org/docs/
- MongoDB Documentation: https://www.mongodb.com/docs/
- Mongoose Documentation: https://mongoosejs.com/docs/
- Multer Documentation: https://github.com/expressjs/multer
- Docker Documentation https://docs.docker.com
- Nodemailer Documentation https://nodemailer.com/about/
- JWT Authentication: https://jwt.io/
- Redis Documentation: https://redis.io/docs/
- Elasticsearch Documentation: https://www.elastic.co/guide/
- Winston Logger Documentation: https://github.com/winstonjs/winston
- GitHub Actions Documentation: https://docs.github.com/actions

2. Overall Description

2.1 Product Perspective

The Blog Management System API is a **new, self-contained backend system** developed independently as a standalone RESTful API. It is not a follow-on to an existing product, nor a replacement for any legacy system currently in use.

The system is designed to function as a **backend subsystem** that can be consumed by external client applications — such as a web frontend, mobile application, or third-party integrations — via well-defined REST API endpoints. While this SRS specifies only the backend components of the system, it is architected to support independent frontend development, since all business logic, data management, and access control are exposed through the API layer.

The system interacts with the following external components:

- **Client Applications** – Any frontend or third-party service that consumes the REST API to perform authentication, content management, and administrative operations.
- **Database Server** – Stores persistent data including users, articles, and comments, with support for replication to ensure scalability and availability.

- **Cache Layer** – Improves performance by reducing redundant database queries for frequently accessed data.
- **CI/CD Pipeline** – Automates testing, building, and deployment of the application to ensure consistent and reliable releases.
- **Load Balancer** – Distributes incoming traffic across multiple instances of the application to maintain performance under load.
- **Cloud Storage Provider** — Stores uploaded article cover images and returns public URLs for retrieval.
- **Email/SMTP Service** – Delivers transactional emails (registration OTP, resend-verification OTP, password-reset OTP) to users; consumed via the system's SMTP integration.

2.2 Product Features

The Blog Management System API provides the following major functional groups. Detailed requirements for each are described in Section 3.

Guest/Public Access

- Unauthenticated guests may access public, read-only content, including published article listings, article details, public comments, and article categories.
- Public article listings support category-based filtering, searching, and sorting.
- Authentication is required for features that create, modify, delete, or otherwise manage user-specific or restricted content.

Authentication & Authorization

- User registration and login, with OTP-based email verification.
- OTP-based password reset flow, with a short-lived reset token (5-minute validity) issued only after OTP verification and delivered via an HttpOnly, Secure cookie.
- Multi-device session support: users may be logged in on multiple devices at the same time, and can log out from the current device, all devices, or a specific device, and view their active sessions.
- Role-based access control distinguishing between registered users and administrators.

Article Management

- Creation, updating, and deletion of blog articles by their respective authors.
- Retrieval of articles for viewing/listing purposes.
- Enforcement of ownership rules, ensuring users can only modify their own content.
- Each article must be assigned to exactly one category; category is available as a filter, search, and sort option on public article listings.
- Cover images are uploaded as files (via Multer) and stored in cloud object storage, rather than submitted as direct URLs.

Comment Management

- Addition of comments on articles by registered users.
- Management and moderation of comments where applicable.
- Support for nested, threaded comment replies.

Category Management

- Full management of article categories (create, update, and status control), restricted to administrators.
- Categories cannot be permanently deleted; administrators mark them active or unavailable instead.
- Public read access to categories for browsing, and for filtering, searching, and sorting articles.

User Management

- Administrative capabilities to manage registered user accounts.
- Ability for administrators to oversee and moderate system-wide content.

System Quality & Operations

- Performance optimization through caching mechanisms.
- System-level logging (API requests/responses, errors, and application events) for monitoring, debugging, and operational auditing via Winston and the ELK stack.
- Automated testing to ensure functional reliability.
- Containerized deployment using Docker for consistency across environments.
- Continuous Integration/Continuous Deployment (CI/CD) for streamlined releases.
- Load balancing and database replication to support scalability and high availability.

2.3 User Classes and Characteristics

The Blog Management System API is designed to serve three primary user classes, differentiated by their privilege levels and the scope of functionality available to them.

Guest (Unauthenticated Visitor)

- Represents any unauthenticated visitor accessing public-facing endpoints without an account.
- Can browse published articles, view article details, view active categories, and view public comments — read-only access with no ability to create, modify, or delete content.
- Not required to have any prior relationship with the system; represents the entry point for user acquisition.

Registered Users (Standard Users)

- Represent the primary and most frequent user class of the system.
- Able to register, log in, and manage their own blog content.
- Can create, update, and delete their own articles, and add comments or replies on articles.
- Do not have access to administrative functions such as managing other users or moderating system-wide content.
- Assumed to have basic technical familiarity, consistent with typical users of a blogging platform (i.e., no specialized technical expertise required).
- This is the **favored user class**, as the core functionality of the system is built around enabling their content creation and interaction workflows.

Administrators

- Represent a smaller, privileged user class responsible for system oversight.
- Possess all permissions available to registered users, in addition to elevated access for managing user accounts, categories, and moderating system content.
- Require a higher level of trust and access control, as their actions affect system-wide data and other users' content.
- Expected to have some familiarity with content moderation and user management responsibilities, though not necessarily deep technical expertise.
- This class is **secondary in frequency of use** compared to standard users, but critical for maintaining system integrity and content quality.

Security and access-control requirements in Section 3 will specify which functions are restricted to Administrators versus available to all Registered Users.

2.4 Operating Environment

The Blog Management System API is built as a backend service intended to run in a containerized, cloud-compatible environment. Its operating environment includes the following components:

Runtime & Framework

- Built on **Node.js**, using the **Express.js** framework for backend application logic and REST API implementation.

Database

- **MongoDB** is used as the primary NoSQL database for persistent data storage, with support for replication (replica sets) to enhance scalability and availability.

File Storage

- **Multer** for handling file uploads; [AWS S3 / Cloudinary] for cloud object storage.

Caching Layer

- A caching system (e.g., Redis or equivalent) is used to reduce database load and improve response times for frequently accessed data.

Containerization & Deployment

- The application is packaged and deployed using **Docker**, ensuring consistent behavior across development, testing, and production environments.
- Deployment is supported by a **CI/CD pipeline**, automating build, test, and release processes.

Infrastructure

- The system is designed to operate behind a **load balancer**, distributing incoming traffic across multiple application instances to maintain performance and availability under load.
- Compatible with standard cloud hosting environments (e.g., AWS, DigitalOcean, or similar Linux-based server environments) that support Docker containers.

Operating System

- The application is platform-independent at the code level (Node.js-based) but is intended to be deployed on **Linux-based server environments**, which is standard for Docker-based production deployments.

2.5 Design and Implementation Constraints

Technology Stack: The system must be implemented using **Node.js** with the **Express.js** framework for the backend, **MongoDB** as the database, **Multer** for handling multipart/form-data file uploads, and cloud object storage (e.g., AWS S3 or Cloudinary) for storing uploaded files.

RESTful Architecture: All client-server communication must follow REST API conventions, using standard HTTP methods (GET, POST, PATCH, DELETE) and status codes.

Authentication Protocol: User authentication must implement industry-standard practices (e.g., JWT-based token authentication) to ensure secure access control.

Containerization: The application must be deployable via **Docker**, requiring all components to be compatible with containerized environments.

Database Design: As MongoDB is a NoSQL database, data modeling must follow document-oriented design principles rather than relational schema conventions.

Security Requirements: Sensitive data such as user passwords must be encrypted/hashed (e.g., using bcrypt) and never stored or transmitted in plain text.

Communication Protocol: All API communication must occur over HTTPS in production environments to ensure data security in transit.

Testing Standards: The system must include automated tests to validate functionality before deployment, as part of the CI/CD pipeline.

Deployment Environment: The system is designed for Linux-based cloud hosting environments compatible with Docker and standard CI/CD tooling.

Token Handling: Refresh tokens and short-lived password-reset tokens must be delivered exclusively via HttpOnly, Secure cookies, and must never be transmitted or accepted via URL path or query parameters.

File Upload Handling: Article cover images are uploaded as files via Multer (multipart/form-data) and stored in cloud object storage; the returned public URL is saved on the article record. Allowed file types are JPG, JPEG, PNG, and WebP, with a maximum file size of 5 MB per image.

2.6 Assumptions and Dependencies

Assumptions

- It is assumed that client applications (web/mobile frontend) consuming this API will be developed independently and are not part of this SRS's scope.
- It is assumed that the hosting environment will provide adequate resources (CPU, memory, storage) to support Docker containers, database replication, and load balancing as the system scales.
- It is assumed that users will access the system over a stable internet connection, as the API does not account for offline functionality.
- It is assumed that MongoDB's replication and scalability features will be sufficient to meet the system's performance goals without requiring a shift to a different database technology.
- It is assumed that OTPs (for email verification and password reset) are delivered via the existing email/SMTP integration (Nodemailer); delivery delays could affect a user's ability to complete verification within the OTP's validity window.

Dependencies

- The system depends on **MongoDB** as a third-party database service, and any changes to its API, licensing, or availability could impact the project.
- The system depends on a third-party cloud storage provider (e.g., AWS S3 or Cloudinary) for storing uploaded files; unavailability, quota limits, or API changes on the provider's side could affect file upload functionality.
- The system depends on external caching solutions (e.g., Redis) for performance optimization; unavailability or misconfiguration of this component could affect response times.
- The project depends on third-party libraries and packages within the Node.js/Express ecosystem (e.g., authentication middleware, validation libraries); changes or deprecations in these dependencies could require code updates.
- The CI/CD pipeline depends on the availability of the chosen CI/CD platform (e.g., GitHub Actions, GitLab CI, Jenkins) for automated testing and deployment.
- The system's scalability features (load balancing, replication) depend on the hosting infrastructure supporting these capabilities.

3. System Requirements

3.1 Functional Requirements

3.1.1 User Registration

- **Description:** Allow new users to create an account.
- **Input:** Name (string), email (string, valid format, case-insensitive), password (string, 8 to 72 bytes).
- **Output:** Success message with user info or error.

- **Process:**
  - Validate input data.
  - Hash the password using bcrypt.
  - Store user data in the database.
  - Send verification OTP to the registered email.
- **Constraints:** Email must be unique. Emails are case-insensitive and are stored in lowercase.

3.1.2 Email Verification

- **Description:** Verify user email using a one-time password (OTP) to activate the account.
- **Input:** Email (string), OTP (string).
- **Output:** Success message or error.
- **Process:**
  - Validate OTP against the OTP stored for the email.
  - Check OTP expiry.
  - Activate user account.
  - Invalidate the used OTP.
- **Constraints:** OTP must be valid and not expired. OTP is single-use.

3.1.3 Resend Verification OTP

- **Description:** Resend a new OTP for email verification.
- **Input:** Email (string).
- **Output:** Success or error message.
- **Process:**
  - Validate email.
  - Invalidate any previous unexpired OTP.
  - Generate new OTP.
  - Send OTP via email.
- **Constraints:** Only unverified users allowed.

3.1.4 User Login

- **Description:** Authenticate user using email and password. Supports multiple simultaneous sessions across different devices.
- **Input:** Email (string), password (string).
- **Output:** Access token and refresh token (refresh token set as an HttpOnly cookie) or error message.
- **Process:**
  - Validate input data.
  - Verify email and password.
  - Generate JWT access token and a device-scoped refresh token.
  - Create a new session record for the device.

- **Constraints:** Only verified and active users can log in. A user may hold multiple active sessions across different devices at the same time.

3.1.5 Token Refresh

- **Description:** Generate a new access token using a valid refresh token for the current device/session.
- **Input:** Refresh token (HttpOnly cookie).
- **Output:** New access token or error message.
- **Process:**
  - Validate refresh token against its corresponding session.
  - Generate new access token.
- **Constraints:** Refresh token must be valid, not expired, and tied to an active session. Refresh tokens expire after 30 days from issuance.

3.1.6 Password Reset Request

- **Description:** Allow users to request a password reset by sending an OTP to their registered email.
- **Input:** Registered email address (string).
- **Output:** Success message or error.
- **Process:**
  - Validate email.
  - Generate OTP.
  - Send OTP via email.
- **Constraints:** Email must be registered in the system.

3.1.7 Verify Password Reset OTP

- **Description:** Verify the OTP sent for password reset and issue a short-lived reset token.
- **Input:** Email (string), OTP (string).
- **Output:** Success message; a short-lived reset token is set as an HttpOnly, Secure cookie (not returned in the response body).
- **Process:**
  - Validate OTP.
  - Invalidate the used OTP.
  - Generate a short-lived reset token.
  - Set the reset token as an HttpOnly, Secure cookie.
- **Constraints:** OTP must be valid and not expired; single-use. Reset token is valid for 5 minutes.

3.1.8 Reset Password

- **Description:** Allow users to set a new password using the reset token issued after OTP verification.

- **Input:** New password (string). The reset token is read from the HttpOnly cookie, not from the request body or URL.
- **Output:** Success message or error.
- **Process:**
  - Read reset token from cookie.
  - Validate reset token.
  - Hash new password using bcrypt.
  - Update password in database.
  - Invalidate reset token and clear the cookie.
- **Constraints:** Reset token must be valid and not expired (5-minute validity); single-use.

3.1.9 User Logout

- **Description:** Log out the user from the current device and invalidate its session.
- **Input:** Refresh token (cookie).
- **Output:** Success message.
- **Process:**
  - Receive logout request.
  - Invalidate the refresh token/session for the current device only.
- **Constraints:** User must be authenticated. Other active sessions on other devices remain unaffected.

3.1.10 Logout from All Devices

- **Description:** Invalidate all active sessions for the user across every device.
- **Input:** None (uses current access token).
- **Output:** Success message.
- **Process:**
  - Identify the authenticated user.
  - Invalidate all refresh tokens/sessions belonging to the user.
- **Constraints:** User must be authenticated.

3.1.11 Logout from Specific Device

- **Description:** Invalidate a single named session/device while leaving other sessions active.
- **Input:** Session/device ID.
- **Output:** Success message or error.
- **Process:**
  - Verify the session belongs to the requesting user.
  - Invalidate that session only.

- **Constraints:** User must be authenticated. Session must belong to the requesting user.

3.1.12 View Active Sessions

- **Description:** Allow users to view their currently active login sessions/devices.
- **Input:** None (uses current access token).
- **Output:** List of active sessions (device/session ID, device info).
- **Process:**
  - Identify the authenticated user.
  - Fetch active sessions for the user.
- **Constraints:** User must be authenticated; users can view only their own sessions.

3.1.13 Change Password

- **Description:** Allow users to update their account password.
- **Input:** User ID, old and new password (string).
- **Output:** Success message or error.
- **Process:**
  - Validate input data.
  - Verify old password.
  - Hash new password using bcrypt.
  - Update password in database.
- **Constraints:** User must be authenticated. An incorrect old password is rejected with 400 Bad Request.

3.1.14 Initial Admin Setup

- **Description:** Create the first administrator account for the system.
- **Input:** Name (string), email (string), password (string).
- **Output:** Success message or error.
- **Process:**
  - Read the adminSetup flag from the SystemInfo record. If it is true, reject the request with an error.
  - Create admin account.
  - Set adminSetup to true in the SystemInfo record..
- **Constraints:** Can only be used once (first-time setup only).

3.1.15 Create Article

- **Description:** Allow authenticated users to create a new article.
- **Input:** Title (string), body (string), cover image (file, multipart/form-data), category (category ID).
- **Output:** Created article or error.

- **Process:**
  - Validate input data.
  - Verify user authentication.
  - Verify category exists and is active.
  - Handle file upload via Multer middleware.
  - Validate file type (JPG, JPEG, PNG, WebP) and size (max 5 MB).
  - Upload file to cloud storage; retrieve the resulting public URL.
  - Save article in database.
- **Constraints:** Only authenticated users can create articles. Category must reference an existing category. Cover image must be JPG, JPEG, PNG, or WebP, and must not exceed 5 MB.

3.1.16 View published Articles

- **Description:** Allow users to retrieve a list of articles.
- **Input:** Pagination parameters (page: positive integer; limit: 1 to 100), sorting (asc, desc, createdAt, updatedAt, title), searching (title), category filter (category ID).
- **Output:** List of articles or empty response.
- **Process:**
  - Fetch articles.
  - Apply sorting, searching, category filtering, and pagination.
- **Constraints:** Only published articles returned.

3.1.17 View Article Details

- **Description:** Retrieve single article.
- **Input:** Article ID, Expand options.
- **Output:** Article data or error.
- **Process:**
  - Retrieve articles from database.
  - Include related data if requested.
- **Constraints:** Article must exist. Public sees only published articles.

3.1.18 View Article Author

- **Description:** Retrieve author of article.
- **Input:** Article ID.
- **Output:** Author details or error.
- **Process:**
  - Fetch article.
  - Return author info.
- **Constraints:** Article must exist. The endpoint returns 404 Not Found for articles in draft status.

3.1.19 Update Article

- **Description:** Allow users to update their own articles.
- **Input:** Article ID, updated fields (title, body, cover image, category).
- **Output:** Success message or error.
- **Process:**
  - Validate input data.
  - Verify user authorization.
  - Verify category exists and is active, if provided.
  - Handle file upload via Multer middleware.
  - Validate file type (JPG, JPEG, PNG, WebP) and size (max 5 MB).
  - Upload file to cloud storage; retrieve the resulting public URL.
  - Update article in database.
- **Constraints:** Only the article owner can update. Cover image must be JPG, JPEG, PNG, or WebP, and must not exceed 5 MB.

3.1.20 Delete Article

- **Description:** Allow users to delete their own articles.
- **Input:** Article ID.
- **Output:** Success message or error.
- **Process:**
  - Verify user authorization.
  - Delete article from database.
  - Remove the article's stored cover image file from cloud storage.
- **Constraints:** Only the article owner or admin can delete. Deleting an article also removes its stored cover image file.

3.1.21 Manage Article Status

- **Description:** Publish or draft the article.
- **Input:** Article ID, Status (published, draft).
- **Output:** Success message or error.
- **Process:**
  - Validate status.
  - Update article status.
- **Constraints:** Only admin can update the status of an article.

3.1.22 View All Articles (Admin)

- **Description:** Retrieve all articles.
- **Input:** Page, Limit, Filters.
- **Output:** List of articles.
- **Process:**
  - Authenticate admin.

    - Fetch all articles.
- **Constraints:** Only admin can access this route.

3.1.23 Create Category

- **Description:** Allow administrators to create a new article category.
- **Input:** Name (string), description (string, optional).
- **Output:** Created category or error.
- **Process:**
    - Validate input data.
    - Verify admin authorization.
    - Check that the category name is unique.
    - Save category in database with active status.
- **Constraints:** Only administrators can create categories. Category name must be unique.

3.1.24 View Categories

- **Description:** Allow users to retrieve a list of categories, for browsing and for filtering, searching, or sorting articles.
- **Input:** Pagination parameters (page: positive integer; limit: 1 to 100), sorting (asc, desc, createdAt, name), searching (name), status filter (administrators only: active, unavailable).
- **Output:** List of active categories (public users) or all categories (administrators), or empty response.
- **Process:**
    - Fetch categories.
    - Apply sorting, searching, status filtering (administrators only), and pagination.
- **Constraints:** Public listing returns only active categories. Administrators can list all categories, including unavailable ones, and can filter them by status.

3.1.25 View Category Details

- **Description:** Retrieve details of a single category.
- **Input:** Category ID.
- **Output:** Category data or error.
- **Process:**
    - Fetch category from database.
- **Constraints:** Category must exist. Public users can view only active categories. Administrators can also view unavailable categories.

3.1.26 Update Category

- **Description:** Allow administrators to update category details.

- **Input:** Category ID, name (string, optional), description (string, optional); at least one of name or description is required.
- **Output:** Success message or error.
- **Process:**
  - Validate input data.
  - Verify admin authorization.
  - Check that the updated name remains unique.
  - Update category in database.
- **Constraints:** Only administrators can update categories. Category name must remain unique. At least one of name or description must be provided.

3.1.27 Manage Category Status

- **Description:** Allow administrators to mark a category as active or unavailable. Categories cannot be permanently deleted.
- **Input:** Category ID, status (active, unavailable).
- **Output:** Success message or error.
- **Process:**
  - Verify admin authorization.
  - Update category status.
- **Constraints:** Only administrators can update status. Unavailable categories are hidden from public listings but remain valid on articles that already reference them.

3.1.28 Create Comment

- **Description:** Allow authenticated users to add a top-level comment to an article.
- **Input:** Article ID, comment text (string).
- **Output:** Created comment or error message.
- **Process:**
  - Validate input data.
  - Verify user authentication.
  - Verify article exists.
  - Store comment in database.
- **Constraints:** User must be authenticated. Article must exist. Created only via the article-scoped comment route.

3.1.29 Reply to Comment

- **Description:** Allow authenticated users to reply to an existing comment on an article, creating a nested/threaded reply.
- **Input:** Article ID and parent comment ID (both from the URL), reply text (string).
- **Output:** Created reply or error message.

- **Process:**
  - Validate input data.
  - Verify user authentication.
  - Verify article and parent comment exist.
  - Store the reply, linked to the parent comment.
- **Constraints:** User must be authenticated. Article and parent comment must exist. Parent comment ID is taken from the URL, not the request body. Replying to a hidden comment is not allowed.

3.1.30 View Article Comments

- **Description:** Allow users to view comments of a specific article.
- **Input:** Article ID, pagination (page: positive integer; limit: 1 to 100).
- **Output:** Comments returned as a nested tree, with each comment including its replies.
- **Process:**
  - Fetch comments for the article.
  - Assemble comments and replies into a nested tree structure.
  - Apply pagination at the top level.
- **Constraints:** Only public comments visible. Article must exist.

3.1.31 Update Comment

- **Description:** Allow users to update their own comments or replies.
- **Input:** Comment ID, updated text (string).
- **Output:** Success message or error.
- **Process:**
  - Validate input data.
  - Verify user authorization.
  - Update comment in database.
- **Constraints:** Only the comment or reply owner can update.

3.1.32 Delete Comment

- **Description:** Allow users to delete their own comments or replies.
- **Input:** Comment ID.
- **Output:** Success message or error.
- **Process:**
  - Verify user authorization.
  - Delete comment (and its replies, if any) from database.
- **Constraints:** Only the comment owner or admin can delete. When a comment is deleted, all of its direct and nested replies must also be deleted.

3.1.33 Manage Comment Status

- **Description:** Hide or show comments.

- **Input:** Comment ID, Status (public, hidden).
- **Output:** Success message or error.
- **Process:**
  - Verify authorization.
  - Update comment status.
- **Constraints:** Only admin can update status.

3.1.34 View All Comments (Admin)

- **Description:** Retrieve all comments.
- **Input:** Filters, Pagination (page: positive integer; limit: 1 to 100).
- **Output:** List of comments or error.
- **Process:**
  - Verify admin authorization.
  - Fetch comments.
- **Constraints:** Only admin can access this route.

3.1.35 Create User

- **Description:** Allow administrators to create new user accounts.
- **Input:** Name (string), email (string, case-insensitive), password (string, 8 to 72 bytes), role (user/admin).
- **Output:** Created user or error message.
- **Process:**
  - Validate input data.
  - Verify admin authorization.
  - Create user account in database.
- **Constraints:** Only administrators can create users. Email must be unique. Emails are case-insensitive and are stored in lowercase. An account created by an administrator shall be assigned account_status = active automatically.

3.1.36 View Users

- **Description:** Allow administrators to view all user accounts.
- **Input:** Filters, Pagination (page: positive integer; limit: 1 to 100).
- **Output:** List of users or empty response.
- **Process:**
  - Verify admin authorization.
  - Retrieve users from database.
- **Constraints:** Only administrators can access user list.

3.1.37 View User Details

- **Description:** Retrieve a single user's details. A registered user may view their own details; an administrator may view any user's details.
- **Input:** User ID, Expand.

- **Output:** User data.
- **Process:**
  - Verify authorization.
  - Fetch user.
- **Constraints:** User must be authenticated. Non-admin users may only retrieve their own record; administrators may retrieve any user's record. User ID must exist.

3.1.38 Update User Information

- **Description:** Allow administrators to update user details.
- **Input:** Name (string), role (string), status (string).
- **Output:** Success message or error.
- **Process:**
  - Validate input data.
  - Verify admin authorization.
  - Update user information in database.
- **Constraints:** Only administrators can update user data. Account status changes must follow the defined User Account Status State Diagram. Administrators may block active users and unblock blocked users. An administrator cannot block or demote their own account; such a request is rejected with 409 Conflict.

3.1.39 Delete User

- **Description:** Allow administrators to delete user accounts.
- **Input:** User ID.
- **Output:** Success message or error.
- **Process:**
  - Verify admin authorization.
  - Delete user from database, together with the user's sessions, OTPs, reset tokens, articles, and comments.
  - Remove the stored cover image files of the deleted articles from cloud storage.
- **Constraints:** Only administrators can delete user accounts. The system shall not allow deletion of the last active administrator. At least one active administrator must remain in the system. An administrator cannot delete their own account. A request that violates this rule or the last-active-administrator rule is rejected with 409 Conflict. Deleting a user also deletes their sessions, OTPs, reset tokens, and articles, every comment on those articles, and every comment written by the user together with all replies beneath it, including replies written by other users. The stored cover image files of the deleted articles are removed as well.

3.2 Non-Functional Requirements

3.2.1 Performance

- **Response Time:** API responses should return within 1-2 seconds under normal load.
- **Scalability:** The system should handle up to 50-100 concurrent users.
- **Throughput:** The system should support at least 300–600 API requests per minute without degradation.

3.2.2 Security

- **Authentication:** JWT for authentication for APIs.
- **Data Protection:** Hash passwords with bcrypt. Encrypt sensitive data in transit (HTTPS).
- **Authorization:** Role-based access control to restrict access to resources based on user roles.
- **Rate limiting:** 5 requests per minute for login and reset password functionality. Extended to also cover the forgot-password, verify-reset-otp, resend-verification-otp, verify-email-otp, sign-up, and setup-admin endpoints (5 requests per minute).
- **Token Delivery:** Refresh tokens and short-lived password-reset tokens are issued only as HttpOnly, Secure cookies, never via URL.
- **OTP Handling:** OTPs are hashed at rest and expire after a short validity window (5 minutes for password reset).
- **File Upload Validation:** Uploaded files are restricted by MIME type and extension whitelist (JPG, JPEG, PNG, WebP), limited to 5 MB, and filenames sanitized before upload to cloud storage.

3.2.3 Reliability

- **Uptime:** 99.9% availability for APIs.
- **Data Integrity:** Ensure consistency of blog posts, comments, and user data during create/update/delete operations. Deleting an article or a user also removes the associated stored cover image files, so that no orphaned files remain in cloud storage.
- **Error Handling:** Proper error handling and logging mechanisms should be implemented to prevent system crashes.

3.2.4 Usability

- **API Design:** APIs should follow RESTful principles with clear and consistent endpoint naming.
- **API Documentation:** Provide API documentation using Swagger/OpenAPI.
- **Developer Experience:** Include clear request/response examples and error messages for easier integration.

3.2.5 Maintainability

- **Modularity:** Follow a layered architecture.
- **Code Quality:** Enforce ESLint and Prettier for consistent code formatting and quality.
- **Testing:** Ensure at least 70–80% unit test coverage for core business logic. Integration tests validate cross-module flows (e.g., authentication, article creation, and commenting working together end-to-end). Load testing validates the system against the throughput and concurrency targets defined in Section 3.2.1 (Performance).
- **Version Control:** Maintain clean Git workflow with proper commit structure.

3.2.6 Portability

- **Deployment:** The system should be deployable on any cloud platform supporting Node.js (e.g., Docker-based environments).
- **Database:** Compatible with MongoDB local and cloud instances.

3.3 Interface Requirements

3.3.1 REST API

- **Endpoints (/api/v1)**
  - POST /auth/sign-up: Register a new user
  - POST /auth/setup-admin: Create initial admin account
  - POST /auth/verify-email-otp: Verify email using OTP
  - POST /auth/resend-verification-otp: Resend email verification OTP
  - POST /auth/sign-in: Log in and receive access token
  - POST /auth/refresh: Refresh access token
  - POST /auth/logout: Log out current device
  - POST /auth/logout-all: Log out all devices
  - DELETE /auth/sessions/{sessionId}: Log out a specific device
  - GET /auth/sessions: List active sessions/devices
  - POST /auth/forgot-password: Request password reset OTP
  - POST /auth/verify-reset-otp: Verify OTP and issue short-lived reset token (cookie)
  - PATCH /auth/reset-password: Reset password using the reset token cookie
  - POST /articles: Create a new article
  - GET /articles: List published articles (supports category filter/search/sort)
  - GET /articles/admin/all: List all articles (admin)
  - GET /articles/{id}: Get article details
  - PATCH /articles/{id}: Update article
  - PATCH /articles/{id}/status: Set article published/draft status (admin)
  - DELETE /articles/{id}: Delete article

- GET /articles/{id}/author: Get article author
- GET /articles/{id}/comments: List article comments (nested tree)
- POST /articles/{id}/comments: Create a top-level comment for article
- POST /articles/{articleId}/comments/{commentId}/replies: Reply to a comment
- POST /categories: Create a category (admin)
- GET /categories: List active categories (admin: all categories, with status filter)
- GET /categories/{id}: Get category details
- PATCH /categories/{id}: Update category (admin)
- PATCH /categories/{id}/status: Set category active/unavailable (admin)
- GET /comments: List all comments (admin)
- PATCH /comments/{id}: Update comment
- PATCH /comments/{id}/status: Hide/show comment (admin)
- DELETE /comments/{id}: Delete comment
- POST /users: Create a user (admin)
- GET /users: List all users (admin)
- GET /users/{id}: Get user details (self, or any user if admin).
- PATCH /users/{id}: Update user
- DELETE /users/{id}: Delete user
- PATCH /users/{id}/change-password: Change user password

4. System Architecture

4.1 Project Structure

The system follows a layered architecture where all components are organized within a single backend application.

- **server/**: Root backend application directory
  - **src/**: Contains the core source code
    - **api/**: Handles request processing and controllers
    - **routes/**: Defines API routes
    - **model/**: Contains database models (Mongoose schemas)
    - **db/**: Database connection configuration
    - **middleware/**: Custom middleware (auth, error handling)
    - **config/**: Application configuration
    - **lib/**: Shared services (e.g., token, email)
    - **utils/**: Helper functions
    - **app.js**: Express app configuration
    - **index.js**: Application entry point
  - .env: Environment variables
  - swagger.yaml: API documentation

- package.json → Dependencies and scripts
- .eslintrc.json → ESLint configuration

4.2 Technology Stack

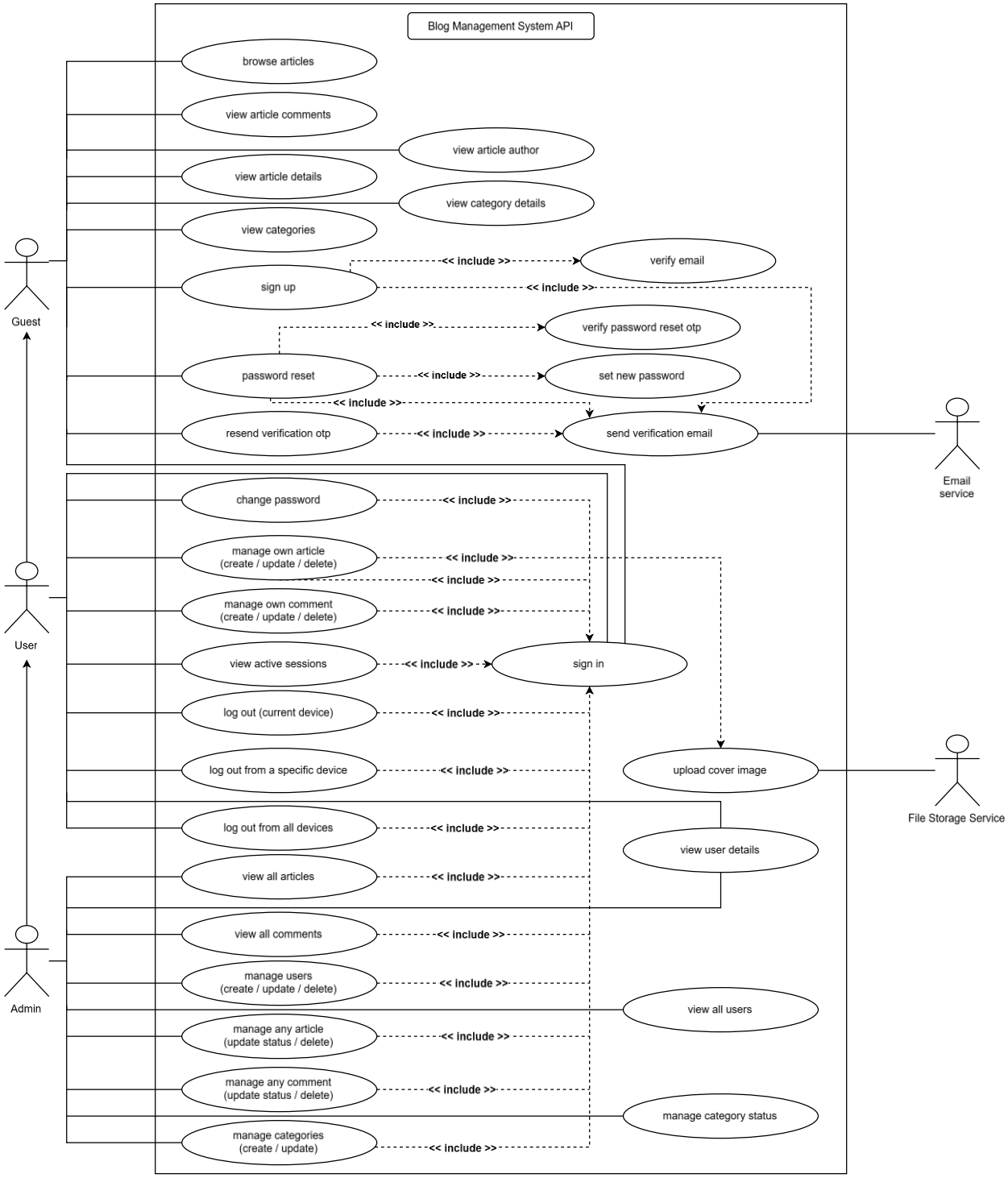
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB, Mongoose (ODM).
- **Authentication & Security:** JWT, bcrypt, HttpOnly Cookies.
- **Email Service:** Nodemailer with SMTP.
- **File Storage:** Multer for handling file uploads; [AWS S3 / Cloudinary] for cloud object storage.
- **API Documentation:** OpenAPI (Swagger YAML).
- **Development Tools:** ESLint, Yarn, Prettier
- **Deployment Tools:** Docker and CI/CD tools for automated build and deployment.
- **Caching:** Redis.
- **Logging:** Winston for structured application logging; **Elasticsearch (ELK stack) for centralized log aggregation.**

Appendix A: Glossary

- **API:** Application Programming Interface.
- **JWT:** JSON Web Token used for authentication.
- **CI/CD:** Continuous Integration / Continuous Deployment.
- **SMTP:** Simple Mail Transfer Protocol used for sending emails.
- **REST:** Representational State Transfer architecture for web services.
- **RBAC:** Role-Based Access Control.
- **OTP:** One-Time Password — a short-lived code used to verify identity for email verification and password reset.
- **Session:** An authenticated login instance tied to a specific device/client, identified by its own refresh token.

Appendix B: Analysis Models

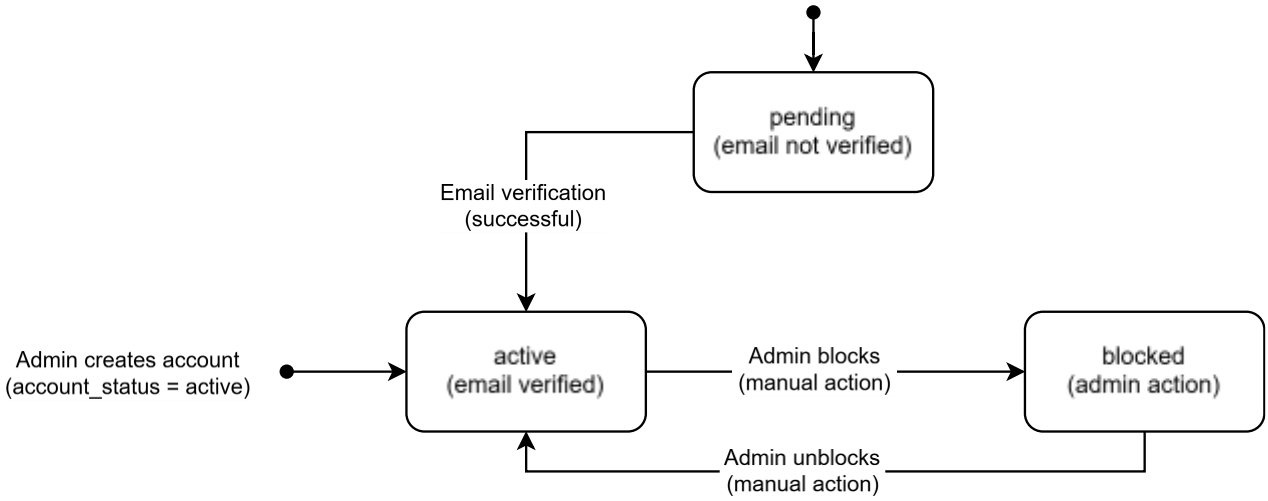
Use Case Diagram



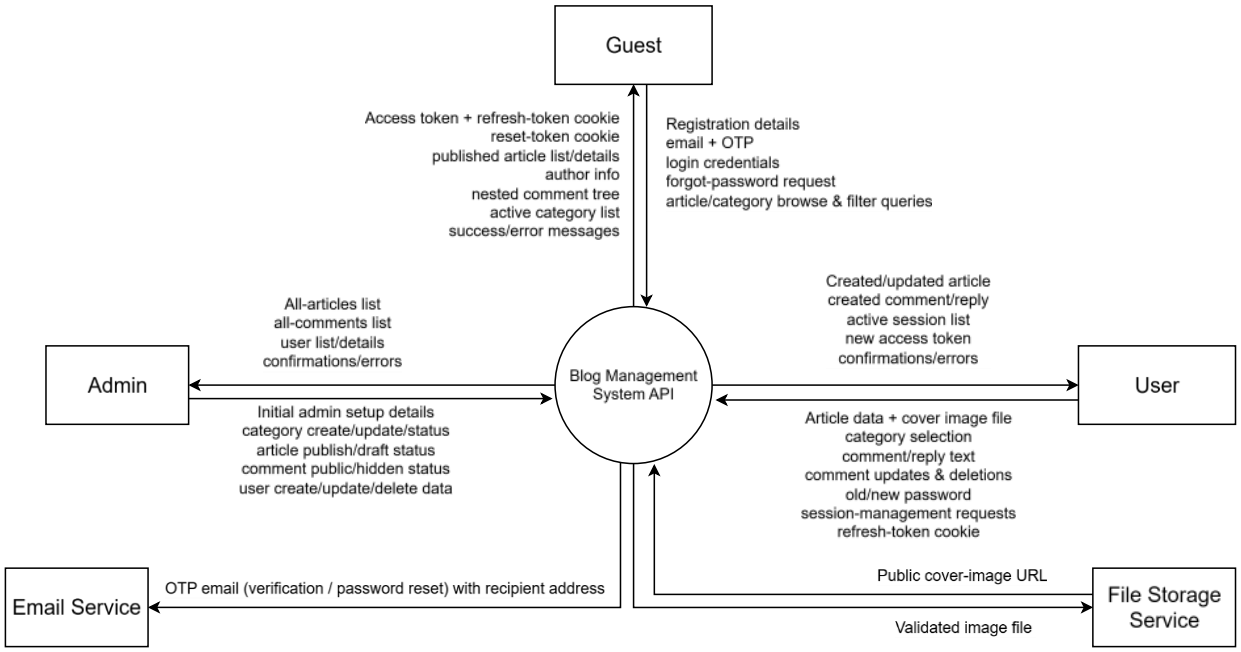
Token refresh (FR-3.1.5): It is not shown as a separate use case because it happens automatically as part of the sign-in/session process. The user does not directly perform this action; the client/application handles it when needed.

Initial Admin Setup (FR-3.1.14): It is not shown as a separate use case because it happens only once when setting up the system for the first time. It is considered a setup/installation task rather than something users normally do while using the application.

Account State Diagram



Data Flow Diagram Level 0



Data Flow Diagram Level 1

